Midterm 2021

Problem 1: (30 pts)

A. (10 pts) Consider the recurrence $T(n) = T(n-2) + 2\log n$. Prove using the substitution method that $T(n) = O(n \log n)$.

Solution:

We guess that $T(n) = \Theta(n \lg n)$. We show the upper bound of $T(n) = O(n \lg n)$ by means of the inductive hypothesis $T(n) \le cn \lg n$ for some constant $c > 0$. We have

$$\begin{aligned} T(n) &= T(n-2) + 2\lg n \\ &\le c(n-2)\lg(n-2) + 2\lg n \\ &\le c(n-2)\lg n + 2\lg n \\ &= (cn - 2c + 2)\lg n \\ &= cn\lg n + (2-2c)\lg n \\ &\le cn\lg n \qquad \text{if } c > 1\,. \end{aligned}$$

Therefore, $T(n) = O(n \lg n)$.

B. (10 pts) Prove or disprove that $3^n = O(2^n)$.

Solution:

Here's a disproof. From the definition of $O(2^n)$ we have that 3^n is only in $O(2^n)$ if there exists some n_0 and c such that $3^n \le c2^n$ for all $n > n_0$. Fix some c. We will find an n for which $3^n > c2^n$. Let $n = 2\lg c$; then $3^n = 3^{2\lg c} = c^{2\lg 3}$, where $2\lg 3 = 3.16992\ldots$, but $c2^n = c \cdot c^2 = c^3 < c^{3.1}$. It follows that if n exceeds $\max(n_0, 2\lg c)$, that $3^n > c2^n$, which proves that 3^n is not $O(n^2)$.

Another disproof uses limits: $\lim_{n\to\infty} \frac{3^n}{2^n} = \lim_{n\to\infty}(3/2)^n$ which diverges. So again 3^n is not in $O(2^n)$.

C. (10 pts) Give the solution in $\Theta(.)$ of each of the recurrences: (apply Master Theorem)

a) $T(n) = 3T\left(\frac{n}{2}\right) + 1$.

b) $T(n) = 3T\left(\frac{n}{2}\right) + n$.

c) $T(n) = 3T\left(\frac{n}{2}\right) + n^2$.

a) $T(n) = O\left(n^{\log_2 3}\right)$; b) $T(n) = O\left(n^{\log_2 3}\right)$; c) $T(n) = n^2$.

a)
Problem 2 (28 pt)
Multiple choice questions:

- Every correct choice adds 4 points
- Every wrong choice adds -4 points
- The total score always remains ≥ 0

The value d depends on the specific question.

Choose one or more true answers from the following multiple-choice questions. No need to add explanations.

A. Consider the following sorting algorithm: We split the array into four equal-size pieces, then recursively sort each piece, and then merge them into one sorted array. The complexity is
a) $\Theta(n^2)$ b) $\Theta(n \log n)$ c) $\Theta(n \log \log n)$ d) none of the above. (Choose one)
Solution: b)

B. Remember that in radix sort the complexity is $\Theta((n+k)d)$. I assume that you are familiar with counting sort and radix sort and what the various parameters represent, otherwise you can re-derive them from scratch with simple arguments.
Choose one or more correct statements:
a) Counting sort has complexity $\Theta(n)$ when k (the maximum value of the keys) grows as $O(n^{1.5})$.
b) Counting sort has complexity $\Theta(n)$ when k is $O(1)$.
c) In radix sort k, d are parameters to be determined and increasing k decreases d.
d) Radix sort is not applicable if the maximum value M of a key is increasing faster than a polynomial in n.
e) Radix sort is asymptotically more efficient than mergesort if M grows $O(n^c)$ but has same complexity if M grows $\Theta(n^{\log n})$.
f) if the keys are sorted, then radix sort takes $O(n)$ time.
Solution: b), c), e)

C. Storing a graph $G = (V, E)$, where $|V| = n, |E| = m$. Choose one or more correct statements:
a) In the adjacency matrix representation of G we use $O(n \log n)$ bits.
b) In the adjacency list representation, we use $O((n+m) \log n)$ bits.
c) A graph is dense when $m = 10n$.
d) A graph is dense when $m = 0.9n^2$.
e) If $m = 0.8n^2$ then we prefer the adjacency list representation.
f) If we use DFS we prefer to represent graphs using adjacency lists.
Solution: b), d), f).

Problem 3. True or False [36 pt] (12 parts)
For each of the following questions, circle either T (True) or F (False). There is no penalty for incorrect answers. Add a simple 2-line explanation that justifies your answer.

(a) T F [3 pt] Given two heaps with n elements each, it is possible to construct a single heap comprising all 2n elements in O(n) time.
Solution: TRUE. Simply traverse each heap and read off all 2n elements into a new array. Then, make the array into a heap in O(n) time by calling MAX- HEAPIFY for i = 2n down to 1.

(b) T F [3 pt] Building a heap with n elements can always be done in O(n log n) time.
Solution: TRUE. In fact, we can build a heap in O(n) time by putting the elements in an array and then calling MAX-HEAPIFY for i = n down to 1.

(c) T F [3 pt] Given a hash table of size n with n elements, using chaining, the minimum element can always be found in O(1) time.
Solution: FALSE. We will need to scan the entire table; this takes Ω(n) time.

(d) T F [3 pt] Running merge sort on an array of size n which is already correctly sorted takes O(n) time.
Solution: FALSE. The merge sort algorithm presented in class always divides and merges the array O(log n) times, so the running time is always O(n log n).

(e) T F [3 pt] We can always find the maximum in a min-heap in O(log n) time.
Solution: FALSE. The *maximum* element in a min-heap can be anywhere in the bottom level of the heap. There are up to n/2 elements in the bottom level, so finding the maximum can take up to O(n) time.

(f) T F [3 pt] In a heap of depth d, there must be at least 2d elements. (Assume the depth of the first element (or root) is zero).
Solution: TRUE. The minimum number of elements in a heap of depth d is one more than the maximum number of elements in a heap of depth (d − 1).
Since a level at depth d in a binary heap can have up to 2d elements, the number d−1 of elements at depth (d − 1) is $\sum_{i=0}^{d-1} 2^i = 2^d - 1$. So the minimum number of elements to increase the depth from n-1 to n is to add one more, i.e., $2^d - 1 + 1 = 2^d$.

(g) T F [3 pt] Consider a family of hash functions consisting of the single hash function $h(k) = k - 99, k = 0,1, \dots, 99$. Suppose that the set of keys $U = \{0, \dots, 10\}$. Is this family of hash functions universal?
Solution: TRUE. For any 2 keys that are different, they are mapped with probability 1 to a different location.

(h) TF [3 pt] Consider the hash function defined earlier, a hash table of size $m = 100$, and keys from $U = \{0, \dots, 10\}$.
Does this hash function satisfy the simple uniform hashing assumption?
Solution: FALSE. The first part of the hash table is never used.

(i) T F [3 pt] Inserting an element into a binary search tree of size n always takes O(log n) time.
Solution: FALSE. Inserting an element into a binary search tree takes O(h) time, where h is the height of the tree. If the tree is not balanced, h may be much larger than log n (as large as n − 1).

(j) T F [3 pt] Any two (possibly unbalanced) BSTs containing n elements each can be merged into a single balanced BST in O(n) time.
Solution: TRUE. Use in-order traversal of the two BSTs to create two sorted lists of length n in O(n) time, merge them into a single sorted list of length 2n in O(n) time, and then create a balanced BST from the sorted lists in O(n) time.

(k) T F [3 pt] Breath First Search (BFS) detects loops in directed graphs in $O(|V| + |E|)$.
Solution: FALSE.

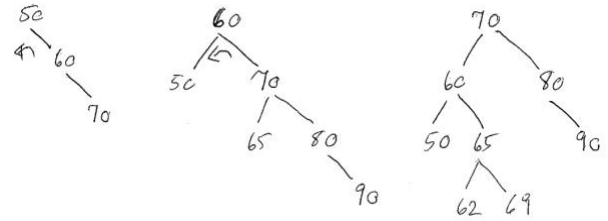
(l) T F [3 pt] BFS detects loops in undirected graphs in $O(\max\{|V|, |E|\})$.
Solution: TRUE.

Problem 4 (30 pt)
A. (10 pt) We are inserting 50,60,70,65,80,90,62,69 into an AVL tree that is initially empty. Show the result after each insertion (after the corresponding balancing operation if it is necessary).

B. (10 pt) Bob give Alice a comparison-based sorting algorithm that runs in $T(n) = 4T\left(\frac{n}{5}\right) + 5n \log\log n$. Is there enough information to know if Bob's algorithm is correct? Y/N. Justify.

C. (10 pt) Same as B) above but $T(n) = 7T\left(\frac{n}{6}\right) + \frac{n}{58}$. Y/N. Justify.

Solution:
A)



B) Using MS theorem, complexity is $\Theta(n \log\log n)$ less than $\Theta(n \log n)$, not possible. Hence YES.
C) Using MS theorem, complexity is $\Theta(n^a)$ where $a > 1$. Hence NO since $n^a = nn^{a-1}$ that grows faster than $\Theta(n \log n)$, and we are not sure if it is correct or not.

Problem 5 (30 pt)
You have been asked by your boss to implement a priority queue type of data structure supporting the following 3 operations: (N is the size of the set of keys)

- insert(x): in time $O(\log N)$,
- fortytwo(): returns the 42nd smallest item in time $O(1)$,
- delfortytwo(): deletes the 42nd smallest item in time $O(\log n)$.

If there are less than 42 items, then we return or delete the largest among them.

Note: In the general case we keep track and return the kth smallest element for some fixed k (in our case $k = 42$). This generalizes the typical priority queue where $k = 1$.

Explain (using pseudocode) how to implement the required functionality using one or more data structures that we have defined in the course. To simplify your code, assume that all operations take place when the system has many items, i.e., $N \geq 43$. You don't have to include the logic of the special case of what happens if $N \leq 42$, and assume that there are always a 42nd and a 43d element in the set and that your data structures are correctly initialized.
Hint: Use two heaps.

```
Solution #1:
Maintain a MaxPQ called firstFortyTwo with 42 items on it,
and a MinPQ called theRest with all the rest of the items.
insert(x):
        firstFortyTwo.insert(x);
        if (firstFortyTwo.size() > 42)
            theRest.insert(firstFortyTwo.delMax());
fortytwo():
        if (firstFortyTwo.size() < 42) // Check may be omitted in answer
            throw new NoSuchElementException("N < 42");
        return firstFortyTwo.max();
delFortyTwo():
        if (firstFortyTwo.size() < 42) // Check may be omitted in answer
            throw new NoSuchElementException("N < 42");
        x = firstFortyTwo.delMax();
        if (!theRest.isEmpty())
            firstFortyTwo.insert(theRest.delMin());
        return x;
```

Problem 6 (20 pt)
Describe an efficient method to merge two balanced BSTs with n elements into a balanced BST. We expect your algorithm to run in $O(n)$ time.

Solution: We can start by doing an in-order walk of both trees concurrently. At each step, we compare the two tree elements and add the smaller one into a list, L, before calling that element's successor. When we finish walking both trees, L will contain a sorted list of elements from both trees. This takes $O(n+n) = O(n)$ total time.

Now, from the sorted list, we want to create a *balanced* binary tree, which is the same problem as described in problem set 2. We can do this by setting the root as the middle element of the list, and letting the first half of the list be its left subtree and the second half be its right subtree (recursively creating the balanced subtrees as well). This also takes $O(n+n) = O(n)$ time.

The total time for this algorithm is therefore $O(n)$.

Problem 7 (20 pt)
Suppose the graph $G = (V, E)$ has edges of length $\in \{1,2\}$. Propose an algorithm that runs in $O(|V| + |E|)$ for creating the shortest path tree.
Bonus question (10 pt): what would the complexity of using this algorithm be as a function of n, m, k when the edge lengths are $\in \{1,2, \dots, k\}$?

Solution: break each edge of length k into k edges of length 1 and add the corresponding new intermediate $k-1$ states for each such edge. Use BFS on the resulting graph. Then the complexity is $O(n + km)$.